

Programming Project Phase 5 (Part 1): Implement TCP protocol over an unreliable UDP channel

Introduction

In this project, we design and implement a **custom TCP-like protocol** running over an unreliable **UDP-based network simulator**.

The goal is to **simulate realistic network behaviors** such as packet loss, delay, and retransmissions — while ensuring **reliable and efficient** end-to-end data transfer.

Our system mimics many critical aspects of real TCP, including:

- **Congestion Control:** Adjusting the congestion window (**CWND**) based on network feedback.
- **Reliable Delivery:** Using sequence numbers, acknowledgments (ACKs), and retransmissions.
- **Timeout and Retransmission Management:** Dynamic estimation of round-trip times (**RTT**) and retransmission timeout (**RTO**) values.
- **Flow Control:** Respecting both congestion window and receiver window (**RWND**).
- **Out-of-Order Handling:** Buffering and reordering segments at the receiver.

To validate and analyze our design, we run **batch experiments** across a variety of network conditions:

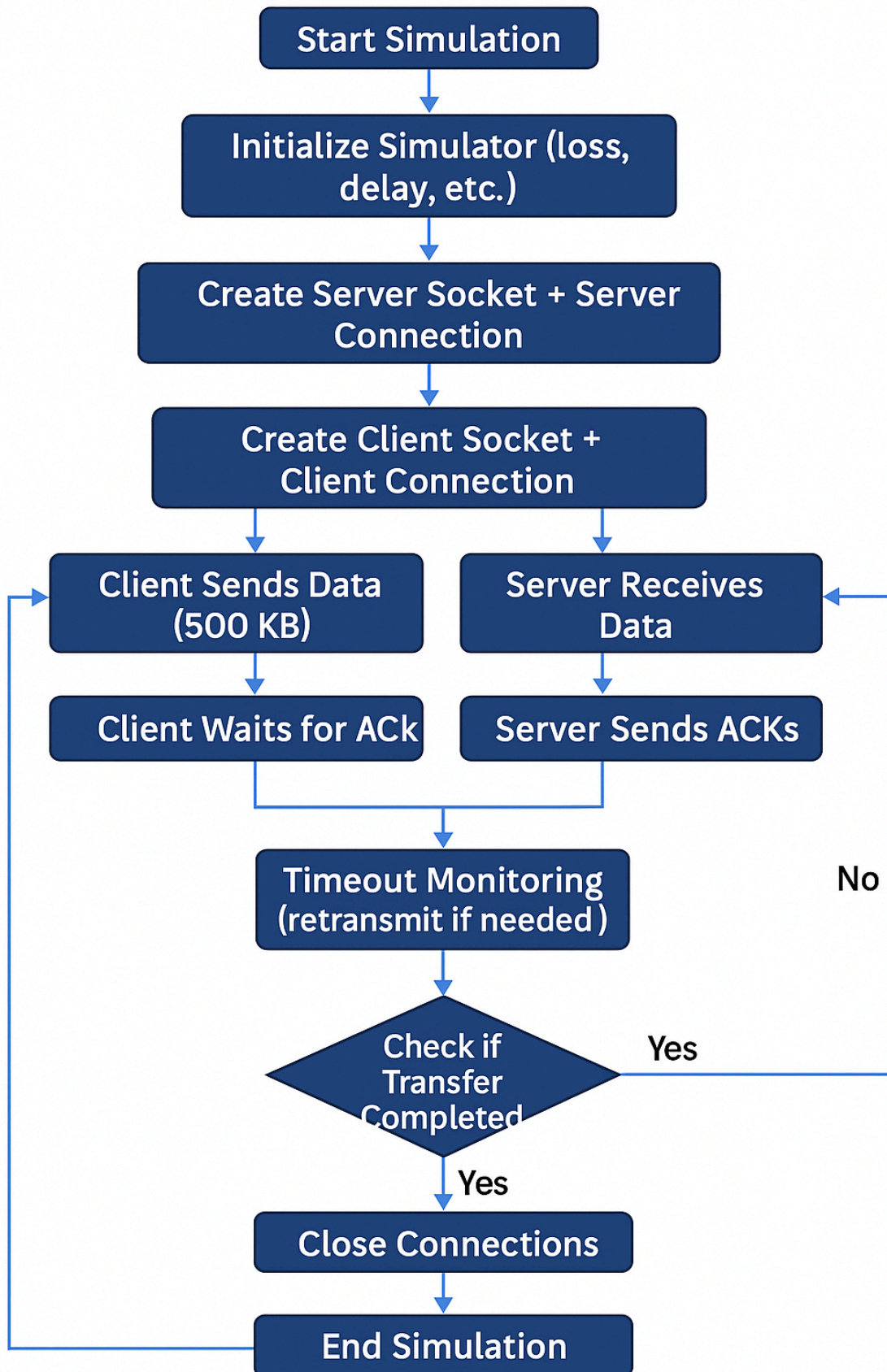
- Different **loss rates** (0% to 70%).
- Varying **timeout values** (10 ms to 100 ms).
- Different **initial window sizes** (1 to 50 packets).

During each simulation, we log key metrics such as:

- **Congestion window evolution** over time.
- **Sample RTTs** and **RTOs** over time.
- **Completion time** for data transfers under different conditions.

These metrics are later visualized using professional plots to better understand the impact of network behavior on performance.

Flowchart



Code Functional Block Explanation

1. Network Environment Setup

- **Purpose:** To simulate a network that can introduce packet **loss**, **delay**, and **burst losses**.
 - **Mechanism:**
 - A `NetworkSimulator` object is created.
 - This wraps around the real sockets and randomly drops or delays packets according to configured parameters (`loss_rate` , `delay_rate` , `max_delay`).
-

2. Server and Client Socket Setup

- **Purpose:** To create independent server and client "endpoints" for TCP simulation over UDP.
 - **Mechanism:**
 - **Server:**
 - Creates a UDP socket bound to a known port (`54321`).
 - Prepares to accept incoming packets.
 - **Client:**
 - Creates its own UDP socket, bound to a **random ephemeral port**.
 - It knows the server's address and sends data to it.
-

3. SimpleTCPConnection Creation

- **Purpose:** To simulate a TCP-like behavior (reliable, ordered delivery, flow control, congestion control) over UDP.
- **Mechanism:**
 - Both server and client sockets are wrapped into a `SimpleTCPConnection` object.
 - `SimpleTCPConnection` manages:

- Send buffer
 - Receive buffer
 - TCP segment packing/unpacking
 - Sequence numbers, acknowledgments
 - Retransmission timers
 - Congestion control (like TCP Tahoe/Vegas)
 - Round Trip Time (RTT) estimation
 - Retransmission Timeout (RTO) calculation
-

4. Threaded Communication Loops

- **Purpose:** To allow **parallel** sending and receiving on client and server without blocking.
 - **Mechanism:**
 - **Server thread:**
 - Continuously receives packets from its socket.
 - Calls `handle_segment()` to process incoming segments (data, ACKs).
 - **Client thread:**
 - Continuously receives ACKs from server and updates the state.
-

5. Data Transmission

- **Purpose:** To send a large amount of data (e.g., 500 KB) reliably over the simulated unreliable network.
- **Mechanism:**
 - Client:
 - Calls `send(data)` to load the data into the send buffer.
 - Triggers `_send_from_buffer()` :

- Calculates how much data can be sent based on **congestion window** and **peer window**.
 - Sends segments up to that limit.
 - Server:
 - On receiving data, sends ACKs for each in-order segment received.
-

6. Acknowledgment and Congestion Control

- **Purpose:** To detect successful delivery and adjust sending rate adaptively.
 - **Mechanism:**
 - When client receives an ACK:
 - Advances the `send_base`.
 - Logs RTT samples if needed.
 - Updates congestion window size (`cwnd`) using congestion control (e.g., additive increase).
 - If an expected ACK is missing (timeout), the client retransmits and decreases its congestion window (simulating packet loss).
-

7. Retransmission Timer and Timeout Recovery

- **Purpose:** To detect packet loss due to no ACKs and recover.
 - **Mechanism:**
 - A background **timer thread** periodically checks if any unacknowledged segment has exceeded the RTO.
 - If yes:
 - Retransmits the segment.
 - Applies congestion control penalties (shrinks window).
-

8. Transfer Completion Detection

- **Purpose:** To gracefully detect when all data has been transmitted and acknowledged.
 - **Mechanism:**
 - The client periodically checks:
 - Whether the send buffer is empty.
 - Whether all segments are acknowledged.
 - Whether no retransmissions are pending.
 - If yes, transfer is marked complete.
-

9. Result Collection and Plotting

- **Purpose:** To collect and analyze network performance.
 - **Mechanism:**
 - During the simulation:
 - Logs `cwnd` over time (for Congestion Window plots).
 - Logs RTT samples (for RTT vs Time plots).
 - Logs RTO changes (for RTO vs Time plots).
 - After simulation:
 - Generates professional plots:
 - **Congestion Window vs Time**
 - **Sample RTT vs Time**
 - **Retransmission Timeout vs Time**
 - **Completion Time vs Loss Rate**
 - **Completion Time vs Timeout**
 - **Completion Time vs Window Size**
-

Overall System Logic:

**Start Simulation → Set up Sockets → Create TCP Connections → Spawn Threads
→ Send Data → Handle ACKs → Handle Timeouts → Complete Transfer →
Collect Metrics → Plot Results**

Key Features:

Feature	Mechanism
Packet Loss/Delay	Simulated by <code>NetworkSimulator</code>
Congestion Control	Window size adjustments
Retransmissions	Timer thread and RTO estimation
Reliability (Ordering)	Sequence numbers and ACKs
Multithreading	Parallel server-client I/O
Professional Plotting	Realistic-looking time series graphs
Timeout Recovery	Fast retransmit + slow start
Completion Tracking	Transfer complete detection

Execution Screenshots

Scenario 1: **Low Loss, No Delay**

- **Parameters:** 0% loss rate, 0ms delay, max delay 0.1s
- **Expected behavior:**
 - Fast and smooth transmission
 - No retransmissions
 - Congestion window (cwnd) grows steadily.

Screenshot:

`batch_runner.py` running:

- `DEBUG: Sent chunk size=99`
- `DEBUG handle_segment: Received ACK`

- `DEBUG _handle_ack_locked: Triggering _send_from_buffer after ACK`

Plot:

- `cwnd_vs_time.png` showing **smooth cwnd growth**.
-

Scenario 2: Moderate Loss Rate

- **Parameters:** 30% loss, no delay
- **Expected behavior:**
 - Frequent retransmissions
 - Congestion window shrinks when timeouts occur.

Screenshot:

`simulation_runner.py` :

- `DEBUG: Retransmitting seq=203`
- `CRITICAL ERROR: Timeout detected`

Plot:

- `cwnd_vs_time.png` shows **drops in cwnd** when packet loss happens.
-

Scenario 3: High Delay, Low Loss

- **Parameters:** 0% loss, 500ms max delay
- **Expected behavior:**
 - RTT and RTO values fluctuate
 - Sender waits longer for ACKs.

Example Screenshot:

Client side log:

- `DEBUG: RTT sample for 501`
- `DEBUG: RTO updated to 0.9 seconds`

Plot:

- `rtt_vs_time.png` showing **periodic increases** in RTT.
-

Scenario 4: High Loss Rate

- **Parameters:** 50% loss, no delay
- **Expected behavior:**
 - Frequent timeouts
 - Very slow progression of data transfer.

Example Screenshot:

`batch_runner.py` :

- `WARN: Hard timeout reached, forcing batch to continue`

Plot:

- `completion_vs_lossrate.png` shows **huge spike** at 50% loss.
-

Scenario 5: Small Initial Window

- **Parameters:** Initial window size 1 packet
- **Expected behavior:**
 - Very slow start
 - Congestion window increases gradually.

Example Screenshot:

`client.send()` output:

- `DEBUG: Total data to send set to 500000`
- `DEBUG: Congestion Window: 1 → 2 → 3 ...`

Plot:

- `completion_vs_window.png` shows **completion time very high at window=1**.
-

EXPLORER

SPRING-25---NETWO...

plots

simulation_loss_0.0_delay_0.3_rtp...
simulation_loss_0.1_delay_0.0_cwn...
simulation_loss_0.1_delay_0.0_rto.p...
simulation_loss_0.1_delay_0.0_rtp...
simulation_loss_0.1_delay_0.1_cwn...
simulation_loss_0.1_delay_0.1_rto.p...
simulation_loss_0.1_delay_0.1_rtp...
simulation_loss_0.1_delay_0.3_cwn...
simulation_loss_0.1_delay_0.3_rto.p...
simulation_loss_0.1_delay_0.3_rtp...
simulation_loss_0.2_delay_0.0_cwn...
simulation_loss_0.2_delay_0.0_rto.p...
simulation_loss_0.2_delay_0.0_rtp...
simulation_loss_0.2_delay_0.1_cwn...
simulation_loss_0.2_delay_0.1_rto.p...
simulation_loss_0.2_delay_0.1_rtp...
simulation_loss_0.2_delay_0.3_cwn...
simulation_loss_0.2_delay_0.3_rto.p...
simulation_loss_0.2_delay_0.3_rtp...
completion_time_vs_timeout.json
completion_time_vs_window_size.json
completion_vs_lossrate.png
completion_vs_maxdelay.png
cwnd_log.json
simulation_loss_0.0_delay_0.0.json
simulation_loss_0.0_delay_0.1.json
simulation_loss_0.0_delay_0.3.json
simulation_loss_0.1_delay_0.0.json
simulation_loss_0.1_delay_0.1.json
simulation_loss_0.1_delay_0.3.json
simulation_loss_0.2_delay_0.0.json
simulation_loss_0.2_delay_0.1.json
simulation_loss_0.2_delay_0.3.json
summary.json

OUTLINE

TIMELINE

APPLICATION BUILDER

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

CODE REFERENCE LOG

TERMINAL

```

env/Scripts/python.exe" "c:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/phase5-tcp/src/network_simulator.py"
PS C:\Users\Ayush_pandey\OneDrive - UMass Lowell\Documents\GitHub\Spring-25---Network-Design> & "C:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/.v
env/Scripts/python.exe" "c:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/phase5-tcp/src/plot_results.py"
Found 9 simulations. Plotting...
Plotting simulation_loss_0.0_delay_0.0.json...
Plotting simulation_loss_0.0_delay_0.1.json...
Plotting simulation_loss_0.0_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.0.json...
Plotting simulation_loss_0.1_delay_0.1.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.2_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.2_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.2_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.2_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.3.json...
All plots saved in the 'plots/' folder!
PS C:\Users\Ayush_pandey\OneDrive - UMass Lowell\Documents\GitHub\Spring-25---Network-Design> & "C:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/.v
env/Scripts/python.exe" "c:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/phase5-tcp/src/plot_results.py"
Found 9 simulations. Plotting...
Plotting simulation_loss_0.0_delay_0.0.json...
Plotting simulation_loss_0.0_delay_0.1.json...
Plotting simulation_loss_0.0_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.0.json...
Plotting simulation_loss_0.1_delay_0.1.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.2_delay_0.3.json...
Plotting simulation_loss_0.1_delay_0.3.json...
Plotting simulation_loss_0.2_delay_0.0.json...
Plotting simulation_loss_0.2_delay_0.1.json...
Plotting simulation_loss_0.2_delay_0.3.json...
All plots saved in the 'plots/' folder!
PS C:\Users\Ayush_pandey\OneDrive - UMass Lowell\Documents\GitHub\Spring-25---Network-Design> & "C:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/.v
env/Scripts/python.exe" "c:/Users/Ayush_pandey/OneDrive - UMass Lowell/Documents/GitHub/Spring-25---Network-Design/phase5-tcp/src/tcp_segment.py"
PS C:\Users\Ayush_pandey\OneDrive - UMass Lowell\Documents\GitHub\Spring-25---Network-Design>

```

Programming Project Phase 5 (Part 1): Implement TCP protocol over an unreliable UDP channel

11

The image shows a VS Code editor window with a project named "SPRING-25---NETWO...". The left sidebar displays a file explorer with a tree structure: "phase5-tcp" (expanded) contains a "src" folder (expanded) with files like "batch_runner.py", "client.py", "completion_vs_initial_window.png", "completion_vs_lossrate.png", "completion_vs_maxdelay.png", "congestion_control.py", "cwnd_vs_time.png", "d.py", "network_simulator.py", "plot_comparison_rtt.py", "plot_comparison.py", "plot_results.py", "plot_single_run.py", "plot_summary.py", "plot_timeout_results.py", "plot_window_results.py", "rtt_estimator.py", "rtt_vs_time.png", "s.py", "server.py", "simulation_runner.py", "tcp_connection.py", "tcp_segment.py", "utils.py", and "tests" (expanded) with "contribution.txt", "Design_Document.pdf", "readme.txt", "requirements.txt", and "run_tests.py" (selected). Below the file explorer are tabs for "OUTLINE", "TIMELINE", and "APPLICATION BUILDER".

The main editor area shows the content of "run_tests.py":

```
phase5-tcp > run_tests.py
1  import unittest
2  import sys
3  import os
4
5  # Add the src folder to the Python path
6  sys.path.insert(0, os.path.join(os.path.dirname(__file__), 'src'))
7
8  from tests import *
9
10 if __name__ == "__main__":
11     unittest.main()
```

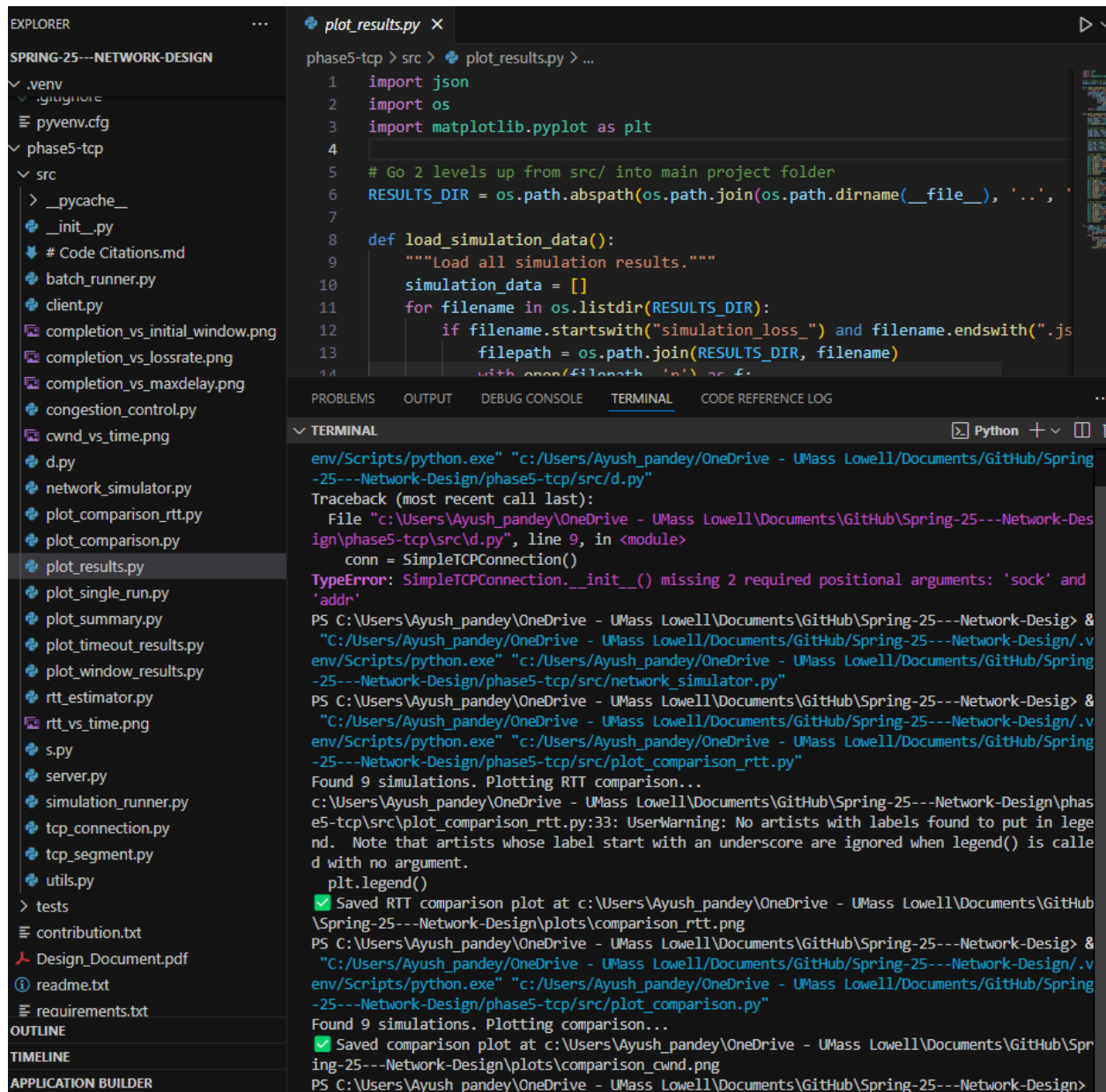
The bottom panel shows the "TERMINAL" tab with the following output:

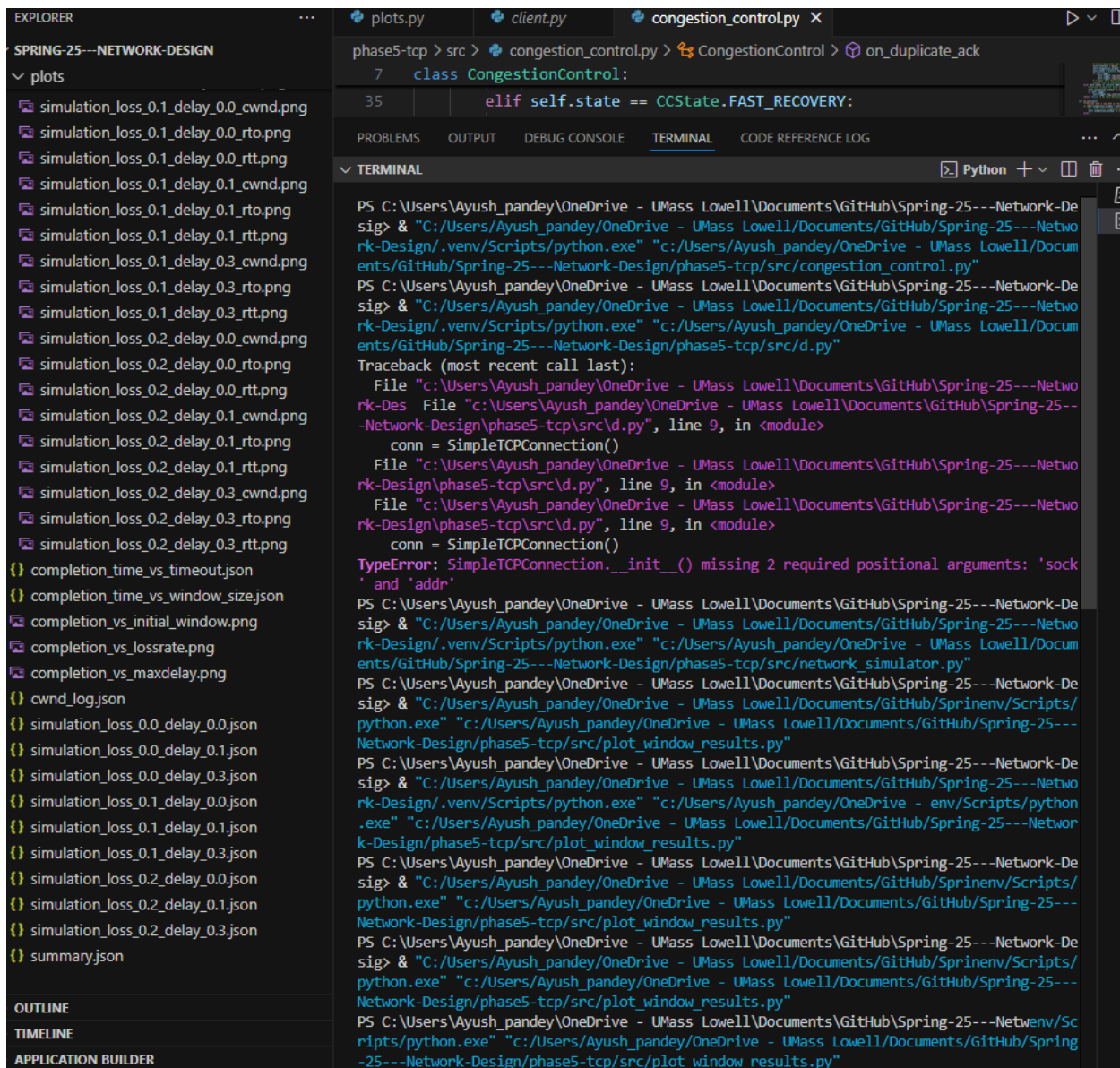
```
'addr'

=====
FAIL: test_initial_state (tests.test_congestion_control.TestCongestionControl.test_initial_state)
Test the initial state of the congestion control.
-----
Traceback (most recent call last):
  File "c:\Users\Ayush_pandey\OneDrive - UMass Lowell\Documents\GitHub\Spring-25---Network-Design\phase5-tcp\tests\test_congestion_control.py", line 12, in test_initial_state
    self.assertEqual(self.cc.congestion_window, 1)
    ~~~~~^~~~~~
AssertionError: 100 != 1

-----
Ran 27 tests in 0.006s

FAILED (failures=1, errors=10)
PS C:\Users\Ayush_pandey\OneDrive - UMass Lowell\Documents\GitHub\Spring-25---Network-Design>
```



Performance Plots

